Lab Handout 3-B
Field-Programmable Gate Array (FPGA) Deployment and Integrated Logic Analyzer (ILA) Debug Configuration

Capstone — Module 3: System Integration and Data Collection

1 Objective

Instrument the Module 2 `ingest_top` design with Vivado ILA probes, generate a debug bitstream, and verify that the ILA dashboard is ready to capture integration failures before any inter-board connections are made. The ILA is the primary diagnostic tool for Module 3; spending 30 minutes configuring it correctly here avoids hours of blind debugging later.

2 Prerequisites

- Lab Handout 3-A complete (all 9 pre-integration checklist items passed).
- Module 2 Vivado project open and `ingest_top.bit` compiling cleanly.

Forward References

This handout mentions LH3-E (production bitstream flow) and occasionally LH3-F through LH3-H (collection, campaigns, dataset handoff). Those later handouts are part of the Module 3 arc and are published alongside this one; if you are reading a draft where they are not yet available, treat the references as pointers to the next milestones rather than prerequisites.

3 Architecture

The debug configuration adds a single ILA core inside a generate-conditional wrapper controlled by a top-level generic. This allows the same Register-Transfer Level (RTL) source to produce either a debug bitstream (with ILA and block-RAM overhead) or a production bitstream (clean, used in LH3-E onwards) without editing the design files.

4 Procedure

4.1 Step 1 — Add the debug generic

1. Open `ingest_top.vhd`. Add a top-level generic:

```
generic (
    g_DEBUG : boolean := true
);
```

[Figure placeholder]

`LH3B_debug_architecture.png`

`ingest_top` debug wrapper. The ILA core is instantiated only when `g_DEBUG = true`. All probe signals are tapped after their source registers to avoid combinational path lengthening.

Figure 1: `ingest_top` debug wrapper. The ILA core is instantiated only when `g_DEBUG = true`. All probe signals are tapped after their source registers to avoid combinational path lengthening.

2. Wrap the ILA instantiation (added in Step 3) in a generate statement:

```
gen_debug : if g_DEBUG generate
    -- ILA core instantiation goes here
end generate gen_debug;
```

3. Leave all existing datapath logic outside the generate block. The generic only gates the ILA, not the pipeline.

Why a generic, not a `define`?

A VHSIC Hardware Description Language (VHDL) generic is evaluated at elaboration time and has no runtime cost. Vivado prunes the generate-false branch entirely before synthesis, so the debug and production builds share identical RTL source and differ only in the top-level instantiation. This is preferable to commenting out ILA lines by hand, which creates version-control noise and is error-prone.

4.2 Step 2 — Probe points

The following table lists the 12 ILA probes as wired in `ingest_top.vhd`. Probe numbers correspond directly to the port map in Step 3.

Probe count

The ILA core is configured with 12 probe ports. Probes 9–11 are tied to `(others => '0')` in the current build; they appear in the dashboard as static zero channels and can be ignored.

Probe	Signal / Source	Width	Purpose
0	`uart_rx_pin`	1	Raw UART input pin; verify serial transitions are reaching FPGA fabric
1	`rx_data`	8	Decoded bytes from UART receiver; validate correct decoding
2	`rx_strobe`	1	Byte-level strobe from `uart_rx`; one pulse per valid byte
3	`fifo_full`	1	FIFO overflow flag; asserts when back-pressure from parser exceeds buffer
4	`frame_valid`	1	Successful double-CRC frame; primary integration milestone trigger
5	`frame_reject`	1	CRC failure; indicates firmware/RTL CRC mismatch
6	`payload(23:16)`	8	Beacon ID (frame byte 2); compare to STM32 log (`0x01`=Target A, `0x02`=Target B)
7	`payload(49:48)`	2	Packet type from low 2 bits of Modulation Code (`00`=DATA, `01`=ACK, `10`=PAUSE)
8	`metadata(39:32)`	8	Parsed RSSI byte; compare to interceptor debug log within ±2 dB

4.3 Step 3 — Insert the ILA core

1. **Open the IP Catalog.** In Vivado's Flow Navigator, click **IP Catalog** (under Project Manager). Search for `ILA` and double-click **ILA (Integrated Logic Analyzer)**.

2. **Configure the core.**

   - **General Options** tab: set **Number of Probes** to `12` and **Sample Data Depth** to `1024`.
   - **Probe_Ports** tab: set probe widths as follows: PROBE0–3 = 1, PROBE4 = 3, PROBE5 = 6, PROBE6–7 = 1, PROBE8 = 8, PROBE9 = 2, PROBE10 = 8, PROBE11 = 6.
   - Click **OK**, then **Generate**. Wait for generation to complete.

3. **Declare the component** in the architecture declaration section of `ingest_top.vhd` (before `begin`):

```
component ila_0 is
port (
    clk     : in std_logic;
    probe0  : in std_logic_vector(0 downto 0);
    probe1  : in std_logic_vector(7 downto 0);
    probe2  : in std_logic_vector(0 downto 0);
    probe3  : in std_logic_vector(0 downto 0);
    probe4  : in std_logic_vector(2 downto 0);
    probe5  : in std_logic_vector(5 downto 0);
    probe6  : in std_logic_vector(0 downto 0);
    probe7  : in std_logic_vector(0 downto 0);
```

```
    probe8  : in std_logic_vector(7 downto 0);
    probe9  : in std_logic_vector(1 downto 0);
    probe10 : in std_logic_vector(7 downto 0);
    probe11 : in std_logic_vector(5 downto 0)
);
end component;
```

4. **Instantiate inside the generate block.** Inside the `gen_debug` block from Step 1, add the port map:

```
u_ila : ila_0
port map (
    clk     => CLK100MHZ,
    probe0  => (0 => uart_rx_pin),
    probe1  => rx_data,
    probe2  => (0 => rx_strobe),
    probe3  => (0 => fifo_full),
    probe4  => (others => '0'),
    probe5  => (others => '0'),
    probe6  => (0 => frame_valid),
    probe7  => (0 => frame_reject),
    probe8  => payload(23 downto 16),
    probe9  => payload(49 downto 48),
    probe10 => metadata(39 downto 32),
    probe11 => (others => '0')
);
```

5. **Verify the generate block boundary.** Confirm only `u_ila` is inside the generate block. All datapath instances (`u_uart`, `u_fifo`, `u_parser`, `u_feat`, `u_bram`) must remain outside.

Block RAM Cost

This configuration (12 probes, 1024-sample depth) uses approximately 4 Block Random Access Memory (BRAM) tiles. The Module 2 pipeline uses 2 BRAM tiles for the feature BRAM. Total with the ILA: 6 of 135 available BRAM tiles on the Artix-7 100T. This is well within budget. The ILA overhead is the primary reason to generate a separate production bitstream in LH3-E once debugging is complete.

4.4 Step 4 — Re-synthesize and implement with debug

1. Confirm that `g_DEBUG` is set to `true` in the top-level instantiation or testbench. In Vivado's **Sources** panel, right-click the top-level and set the generic value.

2. Flow Navigator → **Run Synthesis**. Confirm zero errors. Warnings about ILA probe widths being 1-bit are expected and harmless.

3. Flow Navigator → **Run Implementation**. Confirm positive WNS — the ILA should not introduce timing violations on a 100 MHz design of this size.

4. Flow Navigator → **Generate Bitstream**. The output is `ingest_top.bit` in `impl_1/`; an `.ltx` probes file is generated alongside it.

[Figure placeholder]

`LH3B_vivado_synthesis_ila.png`

Vivado synthesis report showing the ILA core resource usage. The 4 BRAM tiles for the capture buffer should be visible in the utilization section.

Figure 2: Vivado synthesis report showing the ILA core resource usage. The 4 BRAM tiles for the capture buffer should be visible in the utilization section.

4.5 Step 5 — Program the bitstream

1. Connect the Nexys A7 via USB. In Vivado Hardware Manager, program `ingest_top.bit` from `impl_1/`.
2. Vivado will prompt to associate the debug probes file (`ingest_top.ltx`). Click **OK**.
3. The ILA dashboard should open automatically. If it does not, go to **Tools** → **Open Debug Probes** and navigate to `impl_1/ingest_top.ltx`.
4. Confirm that all 12 probe channels are visible in the dashboard.

[Figure placeholder]

`LH3B_ila_dashboard.png`

Vivado ILA dashboard showing all 12 probe channels listed and idle (no UART input connected yet). Confirm the channel names match the signal table before proceeding.

Figure 3: Vivado ILA dashboard showing all 12 probe channels listed and idle (no UART input connected yet). Confirm the channel names match the signal table before proceeding.

4.6 Step 6 — Baseline ILA verification (no STM32 connected)

Do not connect anything to the PMOD header for this step. In the ILA dashboard, signals appear by their name (e.g. `rx_strobe`), not by probe number — probe numbers only appear in the VHDL port map.

1. **Set a trigger on `rx_strobe` rising edge.**
   a. In the ILA dashboard, locate the **Trigger Setup** window (bottom-left panel). If it is not visible, go to **Window** → **Trigger Setup**.
   b. Click the + button to add a trigger condition.
   c. In the signal drop-down, select `rx_strobe`.
   d. Set the **Operator** to `==` and the **Value** to `R` (rising edge). The value field accepts `R`, `F`, `B`, or a hex constant; type `R`.
   e. Click the **Run Trigger** button (the play arrow, or press **F3**). The status indicator changes from *Idle* to *Waiting for Trigger*.
2. **Confirm the trigger never fires.** With no input connected, the status should remain *Waiting for Trigger* indefinitely. If it fires, `uart_rx_pin` is floating and picking up noise. Click **Stop Trigger** (the square stop button, or **F4**), then fix it by adding a pull-up to the `.xdc` constraint file and re-synthesizing:

```
set_property PULLUP true [get_ports uart_rx_pin]
```

3. **Change the trigger to `frame_valid` rising edge and arm again.**
   a. Click **Stop Trigger** if still armed.
   b. In the Trigger Setup window, click the existing condition row. Change the signal to `frame_valid`, operator `==`, value `R`.
   c. Click **Run Trigger**. Confirm it stays in *Waiting for Trigger* with no input connected.
   d. Click **Stop Trigger**.
4. **Capture the idle state of `uart_rx_pin`.**
   a. In the Trigger Setup window, change the signal to `uart_rx_pin`, operator `==`, value `1`.
   b. Click **Run Trigger**. Because the pin is held high at idle, the trigger fires immediately and the waveform viewer fills with samples.
   c. In the waveform viewer, locate the `uart_rx_pin` row. Confirm it shows a flat logic 1 line across all 1024 samples. This confirms the pin is in the correct UART idle state.
   d. Take a screenshot of this waveform — it is required for the deliverable.

ILA Trigger Strategy for Integration Debugging

Use different trigger signals at each debugging layer in LH3-C:

- **No byte activity?** Trigger on `uart_rx_pin` falling edge (start bit). If it never fires, the physical layer is broken.

- **Bytes arrive but no valid frame?** Trigger on `rx_strobe` rising edge and watch `rx_data`. Confirm the first byte captured is `0x7E`; if not, the start delimiter is missing or corrupted.
- **CRC rejects?** Trigger on `frame_reject` rising edge and capture the full `rx_data` sequence for manual CRC computation.
- **Integration success?** Trigger on `frame_valid` rising edge. Compare `beacon_id`, `pkt_type`, and `rssi_out` to the STM32 debug log.

5 Verification Checklist

- ☐ `g_DEBUG` generic and `gen_debug` generate block confirmed in `ingest_top.vhd`.
- ☐ Bitstream synthesizes and implements with zero errors and positive WNS.
- ☐ All 12 ILA probe channels visible in the Vivado dashboard after programming.
- ☐ Baseline test: `rx_strobe` does not assert spuriously with no UART input connected.
- ☐ `uart_rx_pin` idles at logic 1 when PMOD pin is undriven or pulled up.

Lab Handout 3-B Deliverable

1. `ingest_top.vhd` with `g_DEBUG` generic and ILA generate block confirmed.
2. `ingest_top.bit` and `ingest_top.ltx` from `impl_1/`.
3. Post-synthesis utilization screenshot showing ILA BRAM tiles listed.
4. ILA dashboard screenshot with all 12 channels visible and the baseline idle state captured.
5. Note whether a `PULLUP` constraint was added to `uart_rx_pin` in the `.xdc` file.

Glossary

BRAM	Block Random Access Memory	4, 5
CRC	Cyclic Redundancy Check	3, 7
FIFO	First In, First Out	3
FPGA	Field-Programmable Gate Array	1, 3
ILA	Integrated Logic Analyzer	1, 2, 4, 5
RSSI	Received Signal Strength Indicator	3, 7
RTL	Register-Transfer Level	1, 2, 3
UART	Universal Asynchronous Receiver/Transmitter	3, 5, 6, 8
VHDL	VHSIC Hardware Description Language	2, 6